LAB # 6a
Construct various loops in java

Objective

To gain familiarity with the concept in loops.

Lab Goals

To provide working insight into the usage of various loops:

a. Demonstrate the various outputs of different loop.
b. Show effect of casting.
c. Usage of various loops in java.

Introduction to Loops in Java

Looping in programming languages is a feature which facilitates the execution of a set of instructions/functions repeatedly while some condition evaluates to true.

Java provides three ways for executing the loops. While all the ways provide similar basic functionality, they differ in their syntax and condition checking time.

while loop

A while loop is a control flow statement that allows code to be executed repeatedly based on a given Boolean condition. The while loop can be thought of as a repeating if statement.

Syntax

```
while (boolean condition)
{
   loop statements...
}
```

Example#1

```
 public class Main {
    public static void main(String[] args) {
       int i = 1; // Initialize loop control variable
       // While loop to print numbers from 1 to 5
       while (i <= 5) {
          System.out.println(i);
          i++; // Increment the loop control variable
       }
    }
}
```

Output

```
1
2
3
4
5
```

Example#2

```
public class Main {
    public static void main(String[] args) {
        int number = 5; // Number for which factorial is to be calculated
        int factorial = 1;
        int i = 1;
        // While loop to calculate factorial
        while (i <= number) {
            factorial *= i;
            i++;
        }
        System.out.println("Factorial of " + number + " is: " + factorial);
    }
}
```

Output

```
Factorial of 5 is: 120
```

Example#3

```
public class Main {
    public static void main(String[] args) {
        int number = 10; // Upper limit
        int sum = 0;
        int i = 1;
        while (i <= number) {      // While loop to find the sum of even numbers
            if (i % 2 == 0) {
                sum += i;
            }
            i++;
        }
        System.out.println("Sum of even numbers between 1 and " + number + " is: " + sum);
    }
}
```

Output

```
Sum of even numbers between 1 and 10 is: 30
```

for loop

for loop provides a concise way of writing the loop structure. Unlike a while loop, a for statement consumes the initialization, condition and increment/decrement in one line thereby providing a shorter, easy to debug structure of looping

Syntax

```
for (initialization condition; testing condition; increment/decrement) {
   statement(s)
}
```

Example#1

```
public class Main {
    public static void main(String[] args) {
        int sum = 0;
        for (int i = 1; i <= 100; i++) {
            sum += i;
        }
        System.out.println("Sum of numbers from 1 to 100: " + sum);
    }
}
```

Output

```
Sum of numbers from 1 to 100: 5050
```

Example#2

```
 public class Main {
    public static void main(String[] args) {
        int num = 5;
        int factorial = 1;
        for (int i = 1; i <= num; i++) {
            factorial *= i;
        }
        System.out.println("Factorial of " + num + ": " + factorial);
    }
}
```

Output

```
Factorial of 5: 120
```

do while

do while loop is similar to while loop with only difference that it checks for condition after executing the statements, and therefore is an example of **Exit Control Loop**.

Syntax

```
do {
  statements..
}
while (condition);
```

Example#1

```
public class Main {
    public static void main(String[] args) {
        int count = 0;

        // A do-while loop to print numbers from 1 to 5
        do {
            count++;
            System.out.println("Count is: " + count);
        } while (count < 5);
    }
}
```

Output

```
Count is: 1
Count is: 2
Count is: 3
Count is: 4
Count is: 5
```

Example#2

```
public class Main {
    public static void main(String[] args) {
        int n = 10; // Number of Fibonacci numbers to generate
        int first = 0, second = 1;
        int i = 1;
        // Print the first two numbers of the Fibonacci sequence
        System.out.println("Fibonacci Sequence:");
        System.out.print(first + " " + second + " ");
        // Generate and print the rest of the Fibonacci sequence using a do-while loop
        do {
            int next = first + second;
            System.out.print(next + " ");
            first = second;
            second = next;
            i++;
        } while (i < n);
    }
}
```

Output

```
Fibonacci Sequence:
0 1 1 2 3 5 8 13 21 34 55
```

Example#3

```
import java.util.Random;
public class Main {
    public static void main(String[] args) {
        Random random = new Random();
        int rolls = 0, total = 0;
        // Do-while loop to simulate rolling a six-sided die until the total exceeds 20
        System.out.println("Rolling a six-sided die until the total exceeds 20:");
        do {
            int rollResult = random.nextInt(6) + 1; // Generate a random number between 1 and 6
            total += rollResult; // Add the roll result to the total
            rolls++; // Increment the number of rolls
            System.out.println("Roll " + rolls + ": " + rollResult);
        } while (total <= 20);
        System.out.println("Total rolls: " + rolls);
        System.out.println("Total score: " + total);
    }
}
```

Output

```
Rolling a six-sided die until the total exceeds 20:
Roll 1: 4
Roll 2: 1
Roll 3: 2
Roll 4: 1
Roll 5: 5
Roll 6: 6
Roll 7: 1
Roll 8: 6
Total rolls: 8
Total score: 26
```

Sentinel Loop In Java

A sentinel loop in Java is a loop that continues executing until a specific sentinel value is encountered. Here's a short example of a sentinel loop that reads numbers from the user until -1 is entered

Syntax

```
 <initialize loop control variable>;    // Initialization
 // Sentinel loop
 do {
    // Code block to execute repeatedly
    // Check for the sentinel value to terminate the loop
} while (<condition using sentinel value>);
```

Example#1

```
 public class Main {
    public static void main(String[] args) {
       int count = 5;
       // Sentinel loop to count down from 5 to 1
       do {
          System.out.println(count);
          count--;
       } while (count >= 1);

       System.out.println("Blastoff!");
    }
}
```

Output

```
5
4
3
2
1
Blastoff!
```

Nested loops in Java

Nested loops in Java involve placing one loop inside another loop.

Syntax of Nested for Loop in java

```
for (initialization; condition; update) {
    // Outer loop code block

    for (initialization; condition; update) {
        // Inner loop code block
    }
}
```

Syntax of Nested while Loop in java

```
// Outer loop initialization
<outer loop initialization>;

// Outer loop
while (<outer loop condition>) {
    // Inner loop initialization
    <inner loop initialization>;

    // Inner loop
    while (<inner loop condition>) {
        // Code block to execute repeatedly for inner loop

        // Update inner loop control variable(s)
    }

    // Update outer loop control variable(s)
}
```

Example#1 related to Nested while Loop

```
public class Main {
    public static void main(String[] args) {
        int i = 1;

        // Outer while loop for rows
        while (i <= 10) {
            int j = 1;

            // Inner while loop for columns
            while (j <= 10) {
                System.out.print(i * j + "\t");
                j++;
            }

            // Move to the next row
            System.out.println();
            i++;
        }
    }
}
```

Output

```
1       2       3       4       5       6       7       8       9       10
2       4       6       8       10      12      14      16      18      20
3       6       9       12      15      18      21      24      27      30
4       8       12      16      20      24      28      32      36      40
5       10      15      20      25      30      35      40      45      50
6       12      18      24      30      36      42      48      54      60
7       14      21      28      35      42      49      56      63      70
8       16      24      32      40      48      56      64      72      80
9       18      27      36      45      54      63      72      81      90
10      20      30      40      50      60      70      80      90      100
```

Example#2 related to Nested while Loop

```
public class Main {
    public static void main(String[] args) {
        int rows = 5;
        int i = 1;
        // Outer while loop for rows
        while (i <= rows) {
            int j = 1;
            // Inner while loop for printing stars
            while (j <= i) {
                System.out.print("* ");
                j++;
            }
            // Move to the next row
            System.out.println();
            i++;
        }
    }
}
```

Output

Example#3 related to Nested for Loop

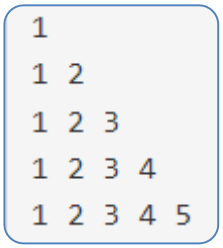
```
public class Main {
    public static void main(String[] args) {
        int size = 5;
        for (int i = 0; i < size; i++) {
            for (int j = 0; j < size; j++) {
                System.out.print("* ");
            }
            System.out.println(); // Move to the next line after each row
        }
    }
}
```

Output

Example#4 related to Nested for Loop

```
public class Main {
    public static void main(String[] args) {
        int rows = 5;
        for (int i = 1; i <= rows; i++) {
            for (int j = 1; j <= i; j++) {
                System.out.print(j + " ");
            }
            System.out.println(); // Move to the next line after each row
        }
    }
}
```

Output



Exercise

A. Write a java program in which print numbers between 20 to 30 using for loop.

B. Write a program in java and print table of 2 using do while loop.

C. Write a program in java and print **star(*)** 100 times using while loop.

Expected Output:

```
**********************************************************************************************
```

D. Write a program to print odd number from 1 to 100 using all three loops.

Expected Output:

```
1 3 5 7 9 ... 97 99
```

E. Write a program to print diamond pattern using nested while and for loop in java.

Expected Output:

```
    *
   ***
  *****
 *******
*********
 *******
  *****
   ***
    *
```

F. Write a program to **reverse a number** using **do-while** loop.

Expected Output:

```
Reversed Number: 4321
```

G. Write a program to find the **sum** and **average** of 10 numbers using **for** loop.

Expected Output:

```
Sum: 55
Average: 5.5
```

H. Write a program to **count** even and odd numbers between 1 and 50 using.

Expected Output:

```
Even numbers: 25
Odd numbers: 25
```

I. Write a program to print the **Hollow square** of star (*****) pattern using **nested for** loop and **conditional statement**.

Expected Output:

```
* * * * *
*       *
*       *
*       *
* * * * *
```